Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 3

Дисциплина МДК 02.02
Сопоставление данных Data Mapping

Вариант № 2
АИС «Электронная медицинская карта»

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Интеграционный сценарий.....3

1.2 Таблица сопоставления.....3

1.3 Ошибки и коллизии.....4

2 ПРОВЕРКА РЕЗУЛЬТАТОВ.....5

ЗАКЛЮЧЕНИЕ.....7

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....8

ПРИЛОЖЕНИЕ А.....9

А.1 Листинг PatientMapper.....9

А.2 Листинг SqlitePatientRepository.....9

А.3 Листинг Program.....11

1 ВЫПОЛНЕНИЕ РАБОТЫ

Цель работы — разработать таблицу сопоставления карточки пациента, правила преобразования типов и обработку коллизий. Работа выполнена по варианту 2 нового сборника [2].

В примерах используются вымышленные пациенты. Оформление выполнено по требованиям института [1].

1.1 Интеграционный сценарий

Реестр отдаёт SourcePatient с текстовой датой dd.MM.yyyy. Внутренняя модель Patient использует DateOnly, код пола M/F/U и время UTC. UUID пациента передаётся из внутренней привязки, а не генерируется заново при каждом импорте.

```
{
  "externalId": "000042",
  "fullName": "  Пациент   Учебный Один ",
  "birthDate": "14.02.1990",
  "sex": "Ж",
  "policyNumber": "0012345678901234",
  "updatedAt": "2026-10-20T10:00:00+03:00"
}
```

1.2 Таблица сопоставления

Сведения представлены в таблице 1.

Таблица 1 — Data Mapping карточки пациента

Источник	Приёмник	Правило
externalId: string	ExternalId: int	1–9 цифр; >0; 000042 → 42
fullName: string	FullName: string	Trim, схлопывание пробелов; 3–150 символов
birthDate: dd.MM.yyyy	BirthDate: DateOnly	Реальная дата; 1900 ≤ дата ≤ 05.11.2026
sex: М/Ж/Н	Sex: M/F/U	Явное сопоставление кодов
policyNumber: string	PolicyNumber: string	Ровно 16 цифр; ведущие нули сохраняются
updatedAt: ISO 8601	UpdatedAt: DateTimeOffset	Обязательное смещение; перевод в UTC
Внутренняя привязка	Id: Guid	Непустой UUID назначения

Источник: составлено по учебному заданию [2].

Номер полиса не преобразуется в число. Это строковый идентификатор, для которого начальные нули значимы. Управляющие символы в имени и неизвестные коды пола отклоняются.

1.3 Ошибки и коллизии

Преобразование завершается до записи в БД. Ошибка поля возвращается как FormatException; частично преобразованная карточка не сохраняется. SQL использует параметры, а изменение выполняется транзакцией.

Сведения представлены в таблице 2.

Таблица 2 — Правила разрешения коллизий

Ситуация	Действие
Новый ExternalId	CREATED
Та же версия и те же данные	DUPLICATE, без новой записи
Старая версия UpdatedAt	STALE, текущая карточка сохраняется
Та же версия, другое содержимое	Конфликт, запись отклоняется
Тот же ExternalId, другой UUID	Конфликт внутренней привязки
Более новая версия	UPDATED с сохранением UUID

Источник: составлено по учебному заданию [2].

UPSERT применяется по уникальному ExternalId. Правила версии проверяются до INSERT/UPDATE, поэтому совпадение ключа не означает безусловную перезапись [3].

2 ПРОВЕРКА РЕЗУЛЬТАТОВ

Программа AIS.MappingCli читает JSON-файл, выполняет PatientMapper.Map и сохраняет результат через SQLite-адаптер. Команда запуска приведена ниже; полный проект находится в отдельном архиве ПР 3.

```
dotnet run --project src/AIS.MappingCli -- data/source-patient.json verification
```

Результат показан на рисунке 1.

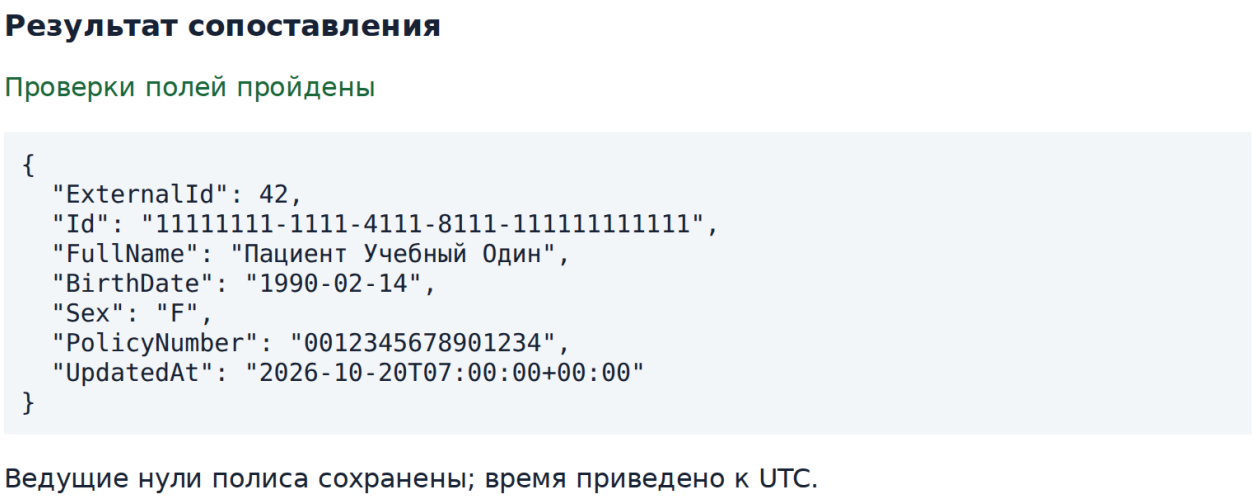


Рисунок 1 — Фактический результат сопоставления карточки

Источник: снимок запущенной программы C#.

Положительные проверки подтверждают сохранение полиса 0012345678901234 и перевод 10:00+03:00 в 07:00 UTC. Отрицательные проверки отклоняют невозможную дату, будущую дату рождения, короткий полис и коллизии версии. Полные листинги приведены в приложении А.

Исходный код, данные и отчёты: https://github.com/rob4ik72-del/MDK0202-EMR

Повторный запуск выполнен на Windows из Visual Studio (рисунок 2). AIS.MappingCli вывел CREATED и нормализованный JSON, процесс завершился с кодом 0. Рабочий каталог профиля запуска — корень решения; входной файл — data/source-patient.json.

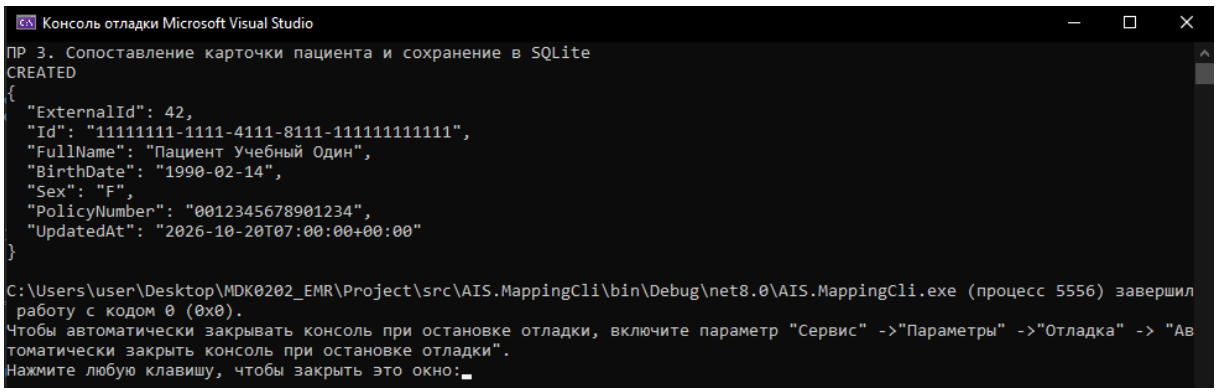


Рисунок 2 — Создание карточки пациента в SQLite при запуске из Visual Studio

ЗАКЛЮЧЕНИЕ

Разработано и реализовано сопоставление SourcePatient → Patient. Различия форматов дат, идентификаторов и кодов пола обрабатываются явно.

Повторная загрузка не создаёт дубль, а конфликт версии не маскируется автоматической перезаписью.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. СПбИЭУ. Положение о требованиях по подготовке и оформлению письменных работ и приложение М: локальный нормативный акт, предоставленный в материалах обучения. [1].

2. МДК 02.02. Детализированный сборник практических работ ПР1–ПР9: электронный учебный материал; вариант 2 «Электронная медицинская карта». [2].

3. SQLite. UPSERT. URL: https://www.sqlite.org/lang_upsert.html (дата обращения: 05.11.2026) [3].

ПРИЛОЖЕНИЕ А

А.1 Листинг PatientMapper

src/AIS.Core/PatientMapper.cs

```
using System.Globalization;
using System.Text.RegularExpressions;

namespace Ais.Core;
public static class PatientMapper
{
   public static Patient Map(SourcePatient source, Guid targetId, DateOnly today)
   {
      if (targetId == Guid.Empty)
         throw new ArgumentException("Пустой ID пациента");
      if (!Regex.IsMatch(source.ExternalId ?? "", @"\A[0-9]{1,9}\z") || !int.TryParse(source.ExternalId,
NumberStyles.None, CultureInfo.InvariantCulture, out int id) || id <= 0)
         throw new FormatException("externalId: положительное целое до 9 цифр");
      if ((source.FullName ?? "").Any(char.IsControl))
         throw new FormatException("fullName: управляющие символы запрещены");
      var name = Regex.Replace((source.FullName ?? "").Trim(), @"\s+", " ");
      if (name.Length < 3 || name.Length > 150 || name.Any(char.IsControl))
         throw new FormatException("fullName: длина 3–150, без управляющих символов");
      if (!DateOnly.TryParseExact(source.BirthDate, "dd.MM.yyyy", CultureInfo.InvariantCulture,
DateTimeStyles.None, out var birth) || birth < new DateOnly(1900, 1, 1) || birth > today)
         throw new FormatException("birthDate: неверная календарная дата");
      string sex = (source.Sex ?? "").Trim().ToUpperInvariant() switch
      {
         "М" => "M",
         "Ж" => "F",
         "Н" => "U",
         _ => throw new FormatException("sex: допустимы М, Ж, Н")};
      if (!Regex.IsMatch(source.PolicyNumber ?? "", @"\A[0-9]{16}\z"))
         throw new FormatException("policyNumber: ровно 16 цифр, строковый тип");
      if (!Regex.IsMatch(source.UpdatedAt ?? "", @"(?:Z|[+-][0-9]{2}:[0-9]{2})\z") || !
DateTimeOffset.TryParse(source.UpdatedAt, CultureInfo.InvariantCulture, DateTimeStyles.None, out var
updated))
         throw new FormatException("updatedAt: ISO 8601 с часовым смещением");
      return new(id, targetId, name, birth, sex, source.PolicyNumber!, updated.ToUniversalTime());
   }
}
```

А.2 Листинг SqlitePatientRepository

src/AIS.Infrastructure/SqlitePatientRepository.cs

```
using Ais.Core;
using Microsoft.Data.Sqlite;
using System.Globalization;

namespace Ais.Infrastructure;
public sealed class SqlitePatientRepository(string path) : IPatientRepository
{
   SqliteConnection Open()
   {
```

```
        var c = new SqliteConnection(new SqliteConnectionStringBuilder { DataSource = path, Pooling = false }.ToString());
        c.Open();
        using var cmd = c.CreateCommand();
        cmd.CommandText = "CREATE TABLE IF NOT EXISTS Patients(ExternalId INTEGER PRIMARY KEY, Id TEXT NOT NULL UNIQUE, FullName TEXT NOT NULL, BirthDate TEXT NOT NULL, Sex TEXT NOT NULL, PolicyNumber TEXT NOT NULL, UpdatedAt TEXT NOT NULL)";
        cmd.ExecuteNonQuery();
        return c;
    }

    static Patient Read(SqliteDataReader r) => new(r.GetInt32(0), Guid.Parse(r.GetString(1)), r.GetString(2), DateOnly.ParseExact(r.GetString(3), "yyyy-MM-dd", CultureInfo.InvariantCulture), r.GetString(4), r.GetString(5), DateTimeOffset.Parse(r.GetString(6), CultureInfo.InvariantCulture));
    public Patient? Find(Guid id)
    {
        using var c = Open();
        using var cmd = c.CreateCommand();
        cmd.CommandText = "SELECT * FROM Patients WHERE Id=$id";
        cmd.Parameters.AddWithValue("$id", id.ToString());
        using var r = cmd.ExecuteReader();
        return r.Read() ? Read(r) : null;
    }

    public string Upsert(Patient patient)
    {
        using var c = Open();
        using var tx = c.BeginTransaction();
        Patient? old;
        using (var cmd = c.CreateCommand())
        {
            cmd.Transaction = tx;
            cmd.CommandText = "SELECT * FROM Patients WHERE ExternalId=$id";
            cmd.Parameters.AddWithValue("$id", patient.ExternalId);
            using var r = cmd.ExecuteReader();
            old = r.Read() ? Read(r) : null;
        }

        if (old != null)
        {
            if (old.Id != patient.Id)
                throw new InvalidOperationException("Конфликт привязки ExternalId к UUID");
            if (old.UpdatedAt > patient.UpdatedAt)
                return "STALE";
            if (old.UpdatedAt == patient.UpdatedAt)
            {
                if (old == patient)
                    return "DUPLICATE";
                throw new InvalidOperationException("Разные данные с одной версией");
            }
        }

        using (var cmd = c.CreateCommand())
        {
            cmd.Transaction = tx;
            cmd.CommandText = "INSERT INTO Patients VALUES($ext,$id,$name,$birth,$sex,$policy,$at) ON CONFLICT(ExternalId) DO UPDATE SET FullName=$name,BirthDate=$birth,Sex=$sex,PolicyNumber=$policy,UpdatedAt=$at";
```

```
            cmd.Parameters.AddWithValue("$ext", patient.ExternalId);
            cmd.Parameters.AddWithValue("$id", patient.Id.ToString());
            cmd.Parameters.AddWithValue("$name", patient.FullName);
            cmd.Parameters.AddWithValue("$birth", patient.BirthDate.ToString("yyyy-MM-dd",
CultureInfo.InvariantCulture));
            cmd.Parameters.AddWithValue("$sex", patient.Sex);
            cmd.Parameters.AddWithValue("$policy", patient.PolicyNumber);
            cmd.Parameters.AddWithValue("$at", patient.UpdatedAt.ToUniversalTime().ToString("O"));
            cmd.ExecuteNonQuery();
        }

        tx.Commit();
        return old == null ? "CREATED" : "UPDATED";
    }
}
```

A.3 Листинг Program

src/AIS.MappingCli/Program.cs

```
using System.Text.Json;
using System.Text.Json.Serialization;
using Ais.Core;
using Ais.Infrastructure;

string input = args.Length > 0 ? Path.GetFullPath(args[0]) : Path.Combine(Environment.CurrentDirectory,
"data/source-patient.json");
string output = args.Length > 1 ? Path.GetFullPath(args[1]) : Path.Combine(Environment.CurrentDirectory,
"verification");
Directory.CreateDirectory(output);
Guid targetId = args.Length > 2 ? Guid.Parse(args[2]) : SampleData.PatientId;
var options = new JsonSerializerOptions(JsonSerializerDefaults.Web)
{
    UnmappedMemberHandling = JsonUnmappedMemberHandling.Disallow
};
var source = JsonSerializer.Deserialize<SourcePatient>(File.ReadAllText(input), options) ?? throw new
FormatException("Пустой JSON источника");
var patient = PatientMapper.Map(source, targetId, new DateOnly(2026, 11, 5));
var repository = new SqlitePatientRepository(Path.Combine(output, "mapping-demo.db"));
Console.WriteLine(repository.Upsert(patient));
var json = JsonSerializer.Serialize(patient, new JsonSerializerOptions { WriteIndented = true, Encoder =
System.Text.Encodings.Web.JavaScriptEncoder.UnsafeRelaxedJsonEscaping });
File.WriteAllText(Path.Combine(output, "mapped-patient.json"), json);
Console.WriteLine(json);
```